
Locks

Locks: The Basic Idea

- To implement a critical section
- A lock variable must be declared
- A lock variable holds the state of the lock
 - Available (unlocked, free)
 - Acquired (locked, held)
- Exactly one thread holds the lock, **owner**
- Once the owner of the lock calls `unlock()`, if there are waiting threads (stuck in `lock()`), one of them will (eventually) notice (or be informed of) this change of the lock's state, acquire the lock, and enter the critical section. *owner가 unlock → lock은 가리키는 thread 중 하나가 lock을 얻음*

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
```

```
Pthread_mutex_lock(&lock); // wrapper for pthread_mutex_lock()  
balance = balance + 1;      } CS  
Pthread_mutex_unlock(&lock);
```

Evaluating Locks

- **Mutual exclusion**
 - whether the lock does its basic task
- **Fairness**
 - Does each thread contending for the lock get a fair shot at acquiring it once it is free?
 - No starvation
- **Performance**
 - time overheads added by using the lock
 - In the case of single thread (no contention)
 - In the case of multiple contending threads/on a single CPU
 - In the case of multiple threads/on multiple CPUs

Controlling Interrupts

- The earliest solutions used to provide mutual exclusion was to disable interrupts for critical sections (single-processor system)
+ Simple *조기미 쓰던 방법*
 - Requires to allow any calling thread to perform a **privileged** operation (interrupt on/off) *user application이 privileged operation을 허용해야 함*
 - Does not work on multiprocessors *multiprocessor에서는 작동X → 다른 processor에서는 여전히 access할 수 있기 때문*
 - Lost interrupts
 - Inefficient: code that masks or unmask interrupts tends to be executed slowly by modern CPUs
- OS itself will use interrupt masking to guarantee atomicity / when accessing its own data structures *OS가 내부적으로 interrupt를 disable하는 경우가 있음*

```
void lock() {  
    DisableInterrupts();  
}  
void unlock() {  
    EnableInterrupts();  
}
```

A Failed Attempt: Just Using Loads/Stores

- Build a simple lock by using a single flag variable

- Correctness problem *정확한 작동 보장*

- Performance problem

- spin-waiting *cpu 낭비*

- high on a uniprocessor

cpu가 쉬지 않을 때 최악적

```
1  typedef struct __lock_t { int flag; } lock_t;
2
3  void init(lock_t *mutex) {
4      // 0 -> lock is available, 1 -> held
5      mutex->flag = 0;
6  }
7
8  void lock(lock_t *mutex) {
9      while (mutex->flag == 1) // TEST the flag
10         ; // spin-wait (do nothing)
11     mutex->flag = 1; // now SET it!
12 }
13
14 void unlock(lock_t *mutex) {
15     mutex->flag = 0;
16 }
```

Thread 1

call lock ()

while (flag == 1) *flag=0*

interrupt: switch to Thread 2

flag를 1로 update 하기 전이

interrupt → T₂에서 call lock()

flag = 1; // set flag to 1 (too!)

틀려 lock하게 됨

Thread 2

call lock ()

while (flag == 1) *flag=0*

flag = 1;

interrupt: switch to Thread 1

Successful but Old Attempt (w/o HW support)

- Dekker's algorithm ('68)
- Peterson's algorithm ('81) P_0 enters CS only if either $\text{flag}[1] = 0$ or $\text{turn} = 0$
- Useless now
 - H/W support for lock and relaxed memory consistency models

```
int flag[2]; (2 threads)
int turn;

void init() {
    flag[0] = flag[1] = 0; // 1->thread wants to grab lock
    turn = 0; // whose turn? (thread 0 or 1?)
}

void lock() {
    flag[self] = 1; // self: thread ID of caller
    turn = 1 - self; // make it other thread's turn
    while ((flag[1-self] == 1) && (turn == 1 - self))
        ; // spin-wait
}

void unlock() {
    flag[self] = 0; // simply undo your intent
}
```

load, store의
순서가 바뀌면
2개의 스레드가
동시에 lock을
획득할 수 있음

load가 reordered될 수 있음 (memory
reorder와 관련 있음)

→ To avoid

상대방이
lock을
획득했는지
확인

상대방이
lock을
획득했는지
확인

상대방의 turn

둘 다 lock을
획득할 때
한쪽만 acquire
하게 됨

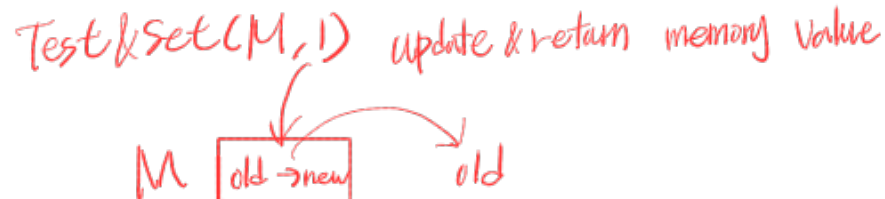
0
↓
lock
flag[0] = 1
turn = 1

Building Working Spin Locks with test-and-set

- We need hardware support!
- **test-and-set** instruction (**atomic exchange**) *→ can not be interrupted*
 - returns the old value pointed to by the `old_ptr` (test), and simultaneously updates said value to `new` (set).
 - this sequence of operations is performed atomically (uninterruptable)

```
int TestAndSet(int *old_ptr, int new) {  
    int old = *old_ptr; // fetch old value at old_ptr  
    *old_ptr = new; // store 'new' into old_ptr  
    return old; // return the old value  
}
```

*Not software function
H/w single instruction
(
uninterruptable*



Building Working Spin Locks with **test-and-set**

- As long as the lock is held by another thread, TestAndSet() will repeatedly return 1, and thus this thread will spin and spin until the lock is finally released.
- By making both the **test** and **set** a single atomic operation, we ensure that only one thread acquires the lock.
- To work correctly on a single processor, it requires a preemptive scheduler

```
1  typedef struct __lock_t {
2      int flag;
3  } lock_t;
4
5  void init(lock_t *lock) {
6      // 0 indicates that lock is available, 1 that it is held
7      lock->flag = 0;
8  }
9
10 void lock(lock_t *lock) {
11     while (TestAndSet(&lock->flag, 1) == 1)
12         ; // spin-wait (do nothing)
13 }
14
15 void unlock(lock_t *lock) {
16     lock->flag = 0;
17 }
```

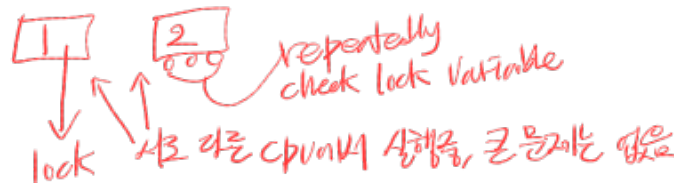
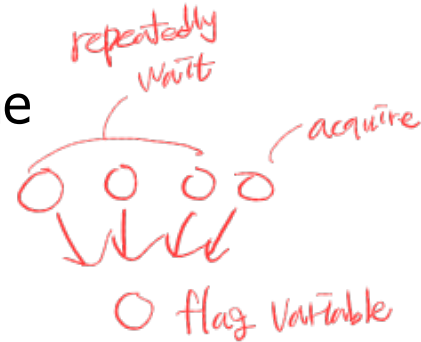
task can be
pre-empted

이걸 1로 바꾸고
다른 스레드가 spin

flag
0 → acquire
1 → 다른 thread가 held → wait

Evaluating Spin Locks

- **Correctness?** Yes
 - allows a single thread to enter CS at a time
- **Fairness?** Bad
 - A thread spinning may spin forever
- **Performance?**
 - Single CPU case: high overhead
 - If the thread holding the lock is pre-empted within a critical section, other scheduled threads try to acquire the lock.
 - Each of those threads will spin for the duration of a time slice before giving up the CPU, a waste of CPU cycles.
 - Multiple CPUs: work reasonably well
 - Thread A on CPU 1 and Thread B on CPU 2, both contending for a lock.
 - Spinning to wait for a lock held on another processor doesn't waste many cycles in this case, and thus can be effective.



Compare-And-Swap (compare-and-exchange)

- Test whether the value at the address specified by `ptr` is equal to expected
 - If so, update the memory location pointed to by `ptr` with the `new` value.
 - If not, do nothing
- Similar behavior but more powerful instruction than test-and-set
 - Useful for lock-free synchronization

Test & set과 유사
↳ compare & swap은 lock,
unlock 및 data
value를 변경 가능

```
int CompareAndSwap(int *ptr, int expected, int new) {  
    int actual = *ptr;  
    if (actual == expected)  
        *ptr = new; 바로 바꿔도  
    return actual; 오래 return  
}
```

```
void lock(lock_t *lock) {  
    while (CompareAndSwap(&lock->flag, 0, 1) == 1)  
        ; // spin  
}
```

0이면 바로 끝

*flag가 0이면
바로 바꿔고 0을
return → spin x*

Load-Linked and Store-Conditional

- load-linked and store-conditional instructions can be used in tandem to build locks and other concurrent structures
- MIPS, Alpha, PowerPC, ARM
- store-conditional only succeeds if no intervening store to the address has taken place

```
int LoadLinked(int *ptr) {  
    return *ptr;  
}  
  
int StoreConditional(int *ptr, int value) {  
    if (no one has updated *ptr since the LoadLinked to this address) {  
        *ptr = value;  
        return 1; // success!  
    } else  
        return 0; // failed to update  
}
```

Handwritten notes:

- load (circled in red) above `LoadLinked`
- store (circled in red) above `StoreConditional`
- Red arrow pointing from the `if` condition to the `LoadLinked` function: *check 다른 process thread update했는지 확인 (write했다면 → fail) 아무도 수정하지 않았다면 success*
- Red note above `return *ptr;`: *네이버로 보기 (flag=1)*

Load-Linked and Store-Conditional

```
void lock(lock_t *lock) {
    while (1) {
        while (LoadLinked(&lock->flag) == 1)
            ; // spin until it's zero
        if (StoreConditional(&lock->flag, 1) == 1)
            return; // if set-it-to-1 was a success: all done
        // otherwise: try it all over again
    }
}

void unlock(lock_t *lock) {
    lock->flag = 0;
}
```

Handwritten notes:
- A red arrow points from the Korean text "다른 프로세스가 업데이트하지 않았는지 check" to the `StoreConditional` function call.
- A red circle is drawn around the `return;` statement, with the word "Success" written below it.

or

```
void lock(lock_t *lock) {
    while (LoadLinked(&lock->flag) || !StoreConditional(&lock->flag, 1))
        ; // spin
}
```

Handwritten notes:
- A red arrow points from the word "fail" to the `||` operator.
- A red arrow points from the word "write" to the `StoreConditional` function call.

Fetch-And-Add

```
int FetchAndAdd(int *ptr) {
    int old = *ptr;
    *ptr = old + 1;
    return old;
}
```

	Ticket	myturn	Turn
Initial	0		0
T1 calls lock	1	0	0
T1 enters CS	1		0
T2 calls lock	2	1	0
T3 calls lock	3	2	0
T1 calls unlock	3		1
T2 enters CS	3		1
T2 calls unlock	3		2
T3 enters CS	3		2
T3 calls unlock	3		3

```
typedef struct __lock_t {
    int ticket;
    int turn;
} lock_t;
```

```
void lock_init(lock_t *lock) {
    lock->ticket = 0;
    lock->turn = 0;
}
```

```
void lock(lock_t *lock) {
    int myturn = FetchAndAdd(&lock->ticket);
    while (lock->turn != myturn)
        ; // spin
}
```

```
void unlock(lock_t *lock) {
    lock->turn = lock->turn + 1; turn++
}
```

Once a thread is assigned its ticket value, it will be scheduled at some point in the future



Ticket locks

high fairness

Too Much Spinning

SpinLock

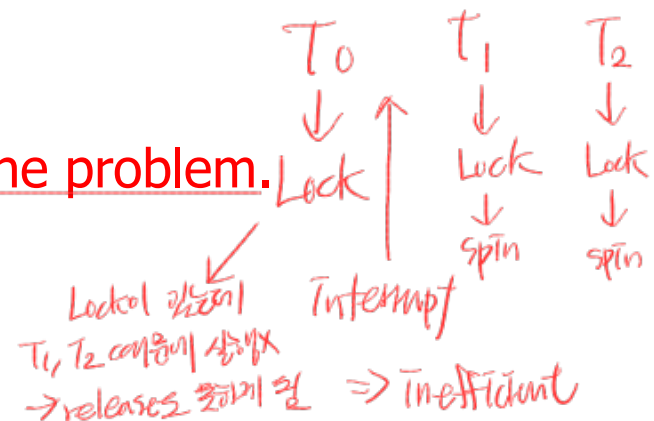
- Hardware-based locks are simple & work
- But, can be quite inefficient
- N threads on a single processor

{ Test&Set
CAS
LLKSC
FKA } hardware로
구현된 것들
→ spin이 너무 많다

- Thread 0 is in a critical section w/ a lock, and unfortunately gets interrupted
- Thread 1 is scheduled and tries to acquire the lock, and it begins to spin.
- Thread 2 is scheduled and tries to acquire the lock, and it begins to spin.
- ...
- N-1 time slices may be wasted

waiting for the
interrupted
(lock-holding)
thread to be run
again

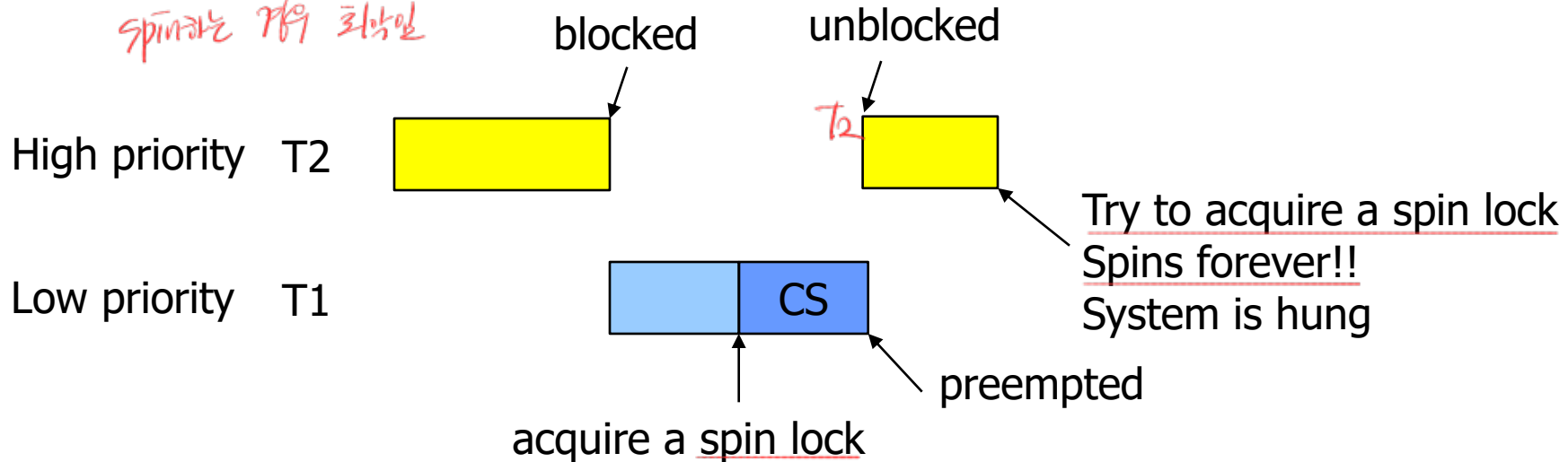
- Hardware support alone cannot solve the problem.
- We'll need OS support too!



More Reason to Avoid Spinning

- Priority-Driven Scheduling

높은 priority를 가진 thread가
spin하는 경우 최악일



How about if just avoid the use of spin locks?

A Simple Approach: Just Yield

- yield system call moves the caller from the running state to the ready state, deschedules itself
- High context switch cost
 - each thread calling lock() will execute run-and-yield pattern before the thread holding the lock gets to run again
 - High context switch cost
- Starvation problem
 - A thread may get caught in an endless yield loop while other threads repeatedly enter and exit the critical section.

```
void init() {  
    flag = 0;  
}  
void lock() {  
    while (TestAndSet(&flag, 1) == 1)  
        yield(); // give up the CPU  
}  
void unlock() {  
    flag = 0;  
}
```

Spinlock: No Context Switching

Using Queues: Sleeping Instead Of Spinning

- Too much left to chance
 - The schedule determines who runs next; if it makes a bad choice – yield immediately or sleep
 - Let's get some control over who gets to acquire the lock next
- Need a queue to keep track of which threads are waiting to acquire the lock.
- A lock puts a caller to sleep if it tries to acquire a held lock and wakes it when the lock is free.

Lock With Queues, Test-and-set, Yield, And Wakeup

```
1 typedef struct __lock_t {
2     int flag;
3     int guard;
4     queue_t *q;
5 } lock_t;
```

```
6
7 void lock_init(lock_t *m) {
8     m->flag = 0;
9     m->guard = 0;
10    queue_init(m->q);
11 }
```

real not variable
guard lock을 치기
start spin (잠들기)

```
12
13 void lock(lock_t *m) {
14     while (TestAndSet(&m->guard, 1) == 1)
15         ; //acquire guard lock by spinning
16     if (m->flag == 0) {
17         m->flag = 1; // lock is acquired
18         m->guard = 0;
19     } else {
20         queue_add(m->q, gettid());
21         m->guard = 0;
22         park();
23     }
24 }
```

Isn't that a race condition?

잠들기

Guard is a spin-lock around the flag and queue manipulations
the time spent spinning is quite limited

Guard lock must be released before park()

put a calling thread to sleep

interrupt로
다른 thread가
unlock하면 문제

```
25
26 void unlock(lock_t *m) {
27     while (TestAndSet(&m->guard, 1) == 1)
28         ; //acquire guard lock by spinning
29     if (queue_empty(m->q))
30         m->flag = 0; // let go of lock; no one wants it
31     else
32         unpark(queue_remove(m->q)); // hold lock (for next thread!)
33     m->guard = 0;
34 }
```

wake a waiting thread; pass the lock directly to the next thread acquiring it; flag is not set to 0 in-between

wakeup/waiting race

- A thread (T0) will be about to park, assuming that it should sleep until the lock is no longer held.
- A switch at that time to the thread (T1) holding the lock.
- T1 releases the lock.
- The subsequent park by T0 would then sleep forever.
- Solaris solution
 - setpark()
 - a thread can indicate it is *about to park*
 - After this, if the thread is interrupted and another calls unpark before the park is called, parks returns immediately
- Another solution
 - pass the guard into the kernel.
 - kernel could take precautions to atomically release the lock and dequeue the running thread.

```
queue_add(m->q, gettid());
setpark(); // new code
m->guard = 0;
park();
```

Different OS, Different Support

- Linux: **futex** (Fast Userspace MuTEX)
 - each futex has associated with it a specific physical memory location, as well as a per-futex in-kernel queue.
 - Provides atomic compare-and-block operation
 - futex is a lower-level construct
 - Used as building blocks for mutex, condition variables, semaphores
 - futex_wait(address, expected)
 - puts the calling thread to sleep if mem[address] = expected
 - If it is not equal, the call returns immediately.
 - futex_wake(address)
 - wakes one thread that is waiting on the queue.
 - Use a 32-bit integer
 - The leftmost bit (the +/- sign) tracks the lock state
 - 0: free, 1: locked
 - Remaining 31 bits: the number of waiters on the lock

Linux-based Futex Locks

glibc's pthread_mutex_lock.c

```
1 void mutex_lock (int *mutex) {
2     int v;
3     /* Bit 31 was clear, we got the mutex (this is the fastpath) */
4     if (atomic_bit_test_set (mutex, 31) == 0) Test the leftmost bit (0: free, 1: locked)
5         return;
6     atomic_increment (mutex); Increment # of waiters
7     while (1) {
8         if (atomic_bit_test_set (mutex, 31) == 0) {
9             atomic_decrement (mutex); After waking up, try to acquire the lock
10            return;
11        }
12        /* We have to wait now. First make sure the futex value
13         we are monitoring is truly negative (i.e. locked). */
14        v = *mutex;
15        if (v >= 0) If lock is free
16            continue;
17        futex_wait (mutex, v); Go to sleep if the lock is not available (*mutex == v)
18    }
19 }
20
21 void mutex_unlock (int *mutex) {
22     /* Adding 0x80000000 to the counter results in 0 if and only if
23      there are not other interested threads */
24     if (atomic_add_zero (mutex, 0x80000000))
25         return;
26     /* There are other threads waiting for this mutex,
27      wake one of them up. */
28     futex_wake (mutex);
29 }
30 }
```

Can acquire lock (points to line 4)

old lock value sleep (points to line 17)

32 (points to bit 31 in the diagram)

0 (points to bit 0 in the diagram)

0000 1111 1111 0000 lock
0000 1111 1111 0000 lockX
No Waiting thread (points to the diagram)

*Add 0x80000000 to *mutex and return 1 if the result is 0*
→ wake threadX (points to line 28)

Two-Phase Locks

- Spinning can be useful, particularly if the lock is about to be released
- **Two-Phase Lock: hybrid between spin-locks and yielding**
- First phase
 - the lock spins for a while, hoping that it can acquire the lock.
 - if the lock is not acquired, a second phase is entered
- Second phase
 - the caller is put to sleep
 - only woken up when the lock becomes free later.

Quiz

1. ~~trylock~~ pthread_mutex_lock() returns immediately if the mutex is locked by another thread. (X)
2. Hardware spinlocks such as test-and-set and compare-and-swap prevent ~~busy-waiting by putting threads to sleep when the lock is held~~ ^{wait}. (X)
3. Spinlocks are more efficient than mutexes when the wait time is expected to be short. (O)
4. Hardware spinlocks always ~~guarantee~~ fairness (FIFO order) among waiting threads. (X)
5. Compare-and-swap updates a memory location only if its current value matches an expected old value. (O)
6. If the mutex is already locked, the calling thread is placed in a wait queue. (O)
7. Using spinlocks instead of mutexes prevents priority inversion. (X)
8. When a thread uses futex_wait, it is added to the futex wait queue inside the kernel if the lock is not free. (O)
9. A futex is just an integer in user space that follows a specific protocol understood by both user code and the kernel. (O)
10. Both mutexes and ~~spinlocks~~ require kernel support for their implementation. (X)

Atomic Instructions in RISC-V

- **LR/SC (Load-Reserved / Store-Conditional) Pair**

- lr.w Load word and reserve
- sc.w Conditionally store if reservation is valid

```
# a0: address of lock variable
# Returns a0 = 0 if lock was free (successfully acquired), non-zero otherwise

test_and_set:
    li      t1, 1           # t1 = 1 (value to write if lock is free)
1:
    lr.w    t0, (a0)        # load-reserved: t0 = *a0
    bnez    t0, fail        # if t0 != 0, lock is already taken
    sc.w    t2, t1, (a0)    # store-conditional: try to store t1
    bnez    t2, 1b          # if store failed (t2 != 0), retry
    li      a0, 0           # success
    ret
fail:
    li      a0, 1           # failed to acquire lock
    ret
```


Atomic Instructions in RISC-V

- **AMOs (Atomic Memory Operations)**
 - Atomic read-modify-write

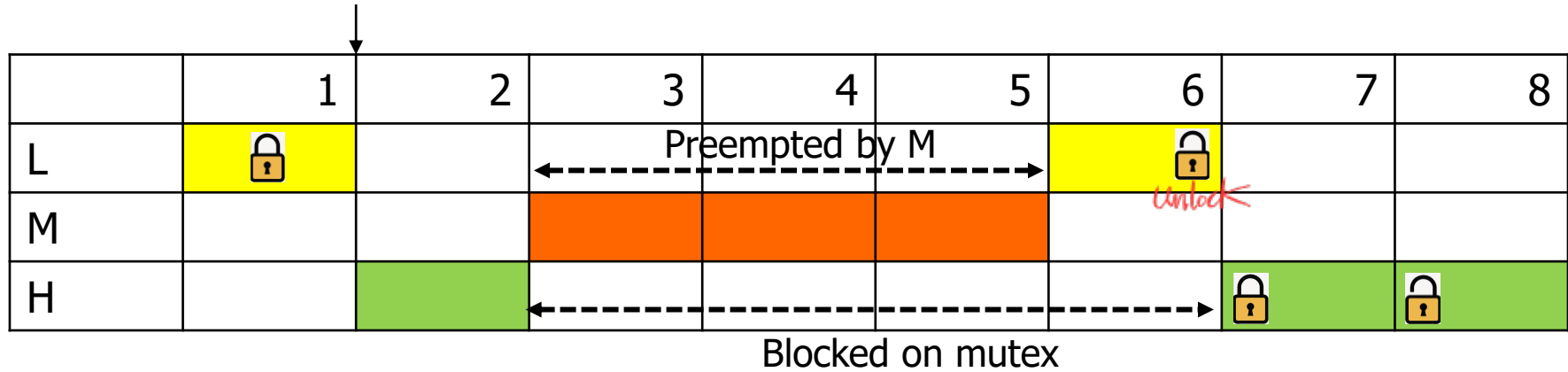
Instruction	Operation	Meaning
amoswap.w	t = *addr; *addr = rs2	Atomic swap


```
test_and_set:
    li    t0, 1
1:
    amoswap.w.aq t1, t0, (a0) # t1 = old lock value, attempt to set to 1
    bnez t1, 1b              # if lock was not 0, retry
```

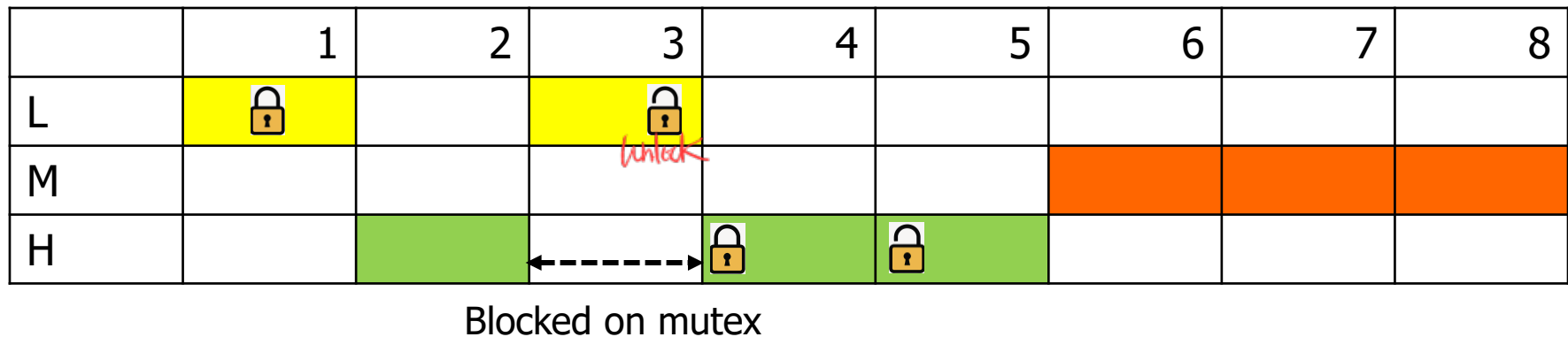
스핀의 preempt X

Priority Inversion

H and M arrive



Without Priority Inheritance



With Priority Inheritance

Priority Inheritance

```
struct mutex {
    thread_t *owner;
    list_t wait_queue;
};

// mutex_lock
if (mutex->owner == NULL) {
    mutex->owner = current_thread;
} else {
    // Block current thread
    add_to_wait_queue(&mutex->wait_queue,
                    current_thread);

    // Priority inheritance
    if (current_thread->priority >
        mutex->owner->priority) {
        boost_priority(mutex->owner,
            current_thread->priority);
    }
    block(current_thread);
}
```

```
// mutex_unlock
mutex->owner = NULL;

if (!is_empty(&mutex->wait_queue)) {
    thread_t *next =
        remove_highest_priority(&mutex->wait_queue);
    mutex->owner = next;
    unblock(next);

    // Restore priority of releasing thread
    restore_original_priority(current_thread);
}
```

```
// pthread programming
pthread_mutex_t mutex;
pthread_mutexattr_t mutex_attr;

pthread_mutexattr_init(&mutex_attr);
pthread_mutexattr_setprotocol(&mutex_attr,
                              PTHREAD_PRIO_INHERIT);
pthread_mutex_init(&mutex, &mutex_attr);
```